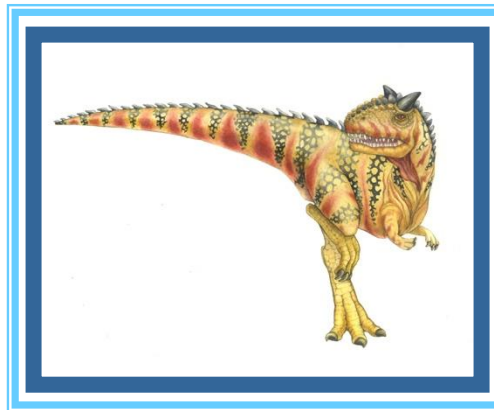


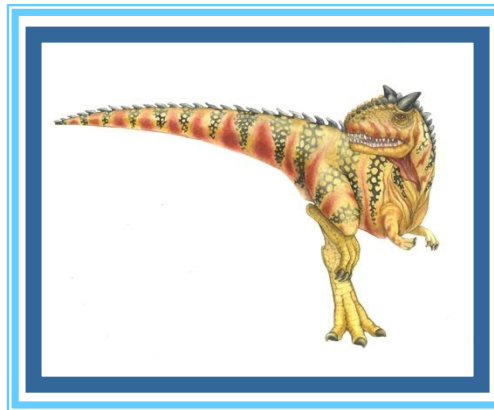
These slides have been modified from the original



A huge thanks to the
authors of Operating
System Concepts for
providing this resource



These slides have been modified from the original



An indicator on a slide means I've modified something on the slide from the original slides

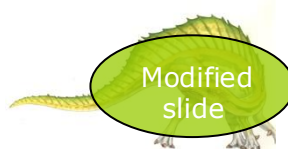
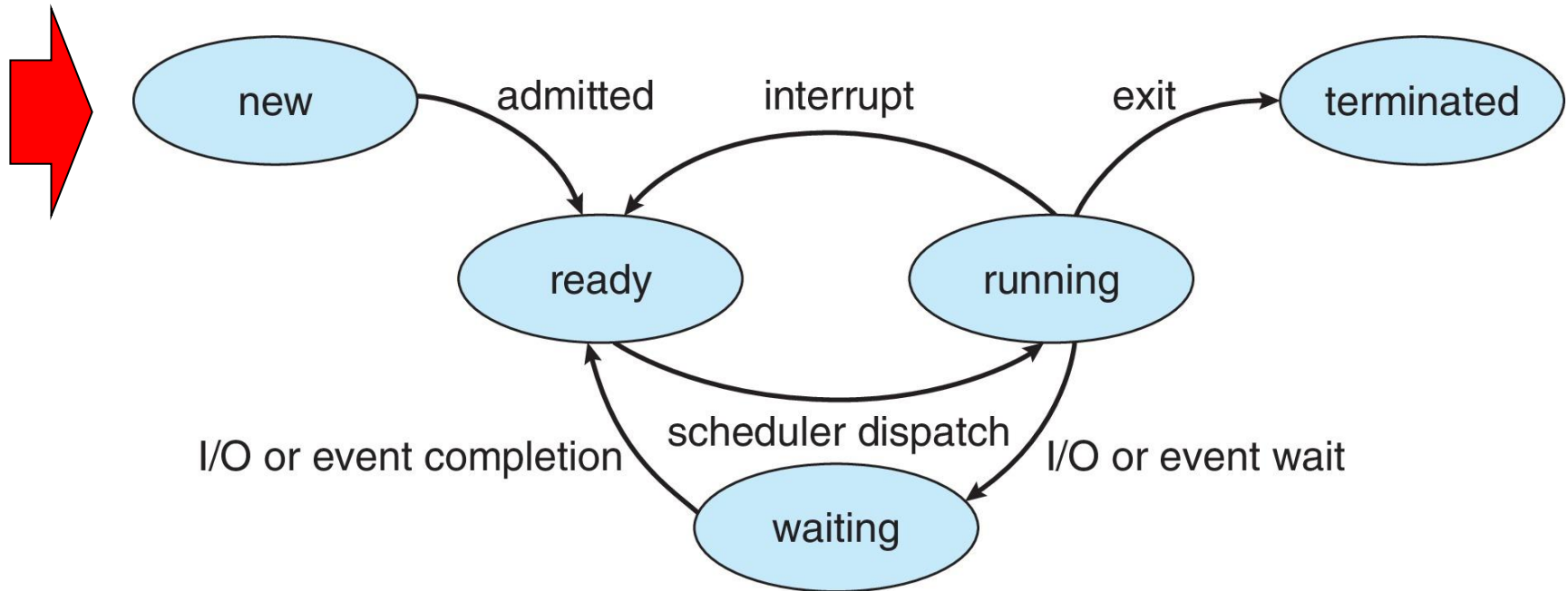
Modified slide

CPU Scheduling



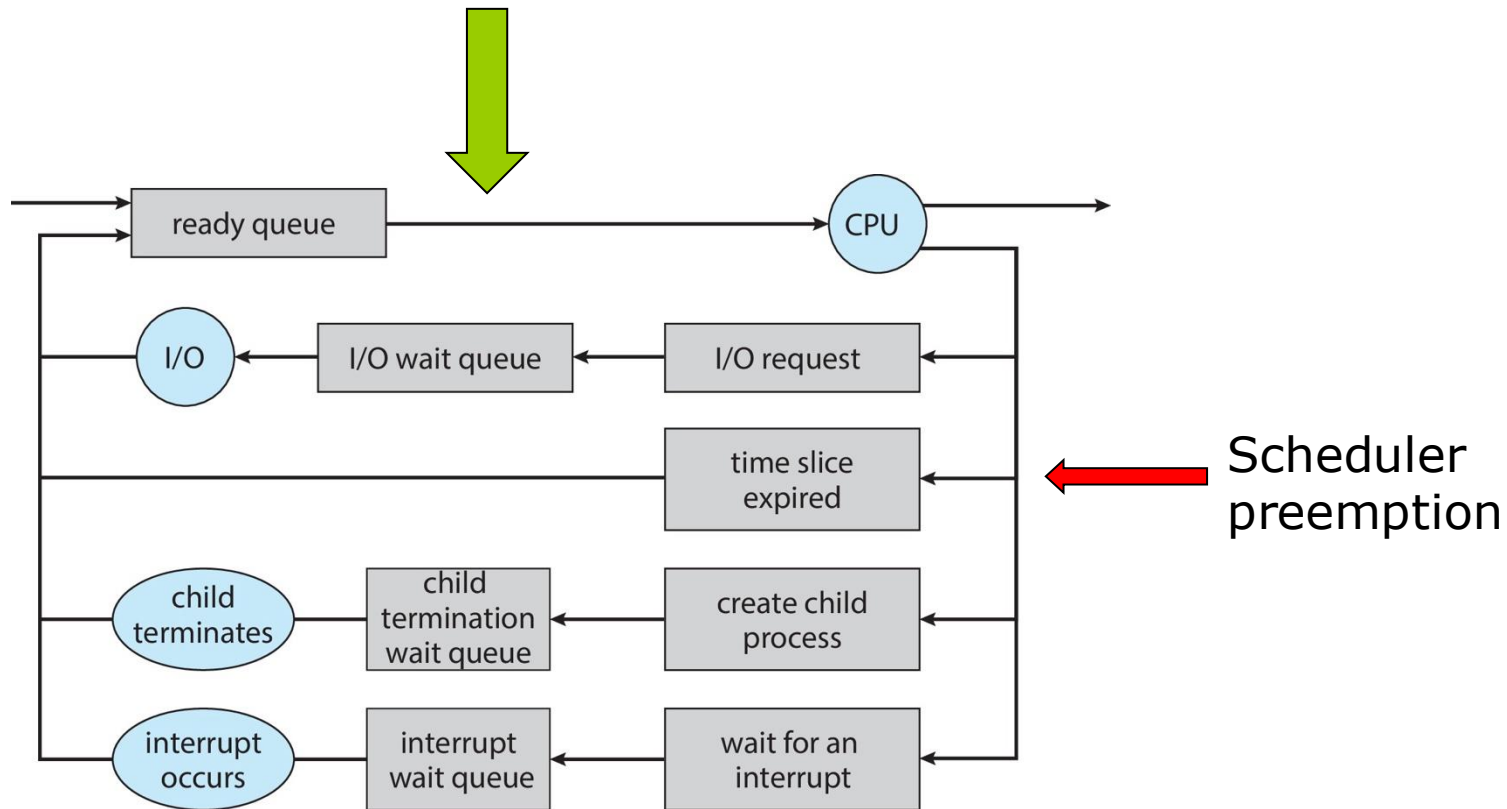
Diagram of Process State

The scheduler operates when processes move between READY and RUNNING.





Representation of Process Scheduling



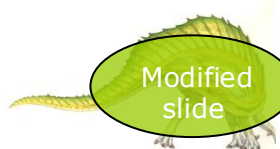
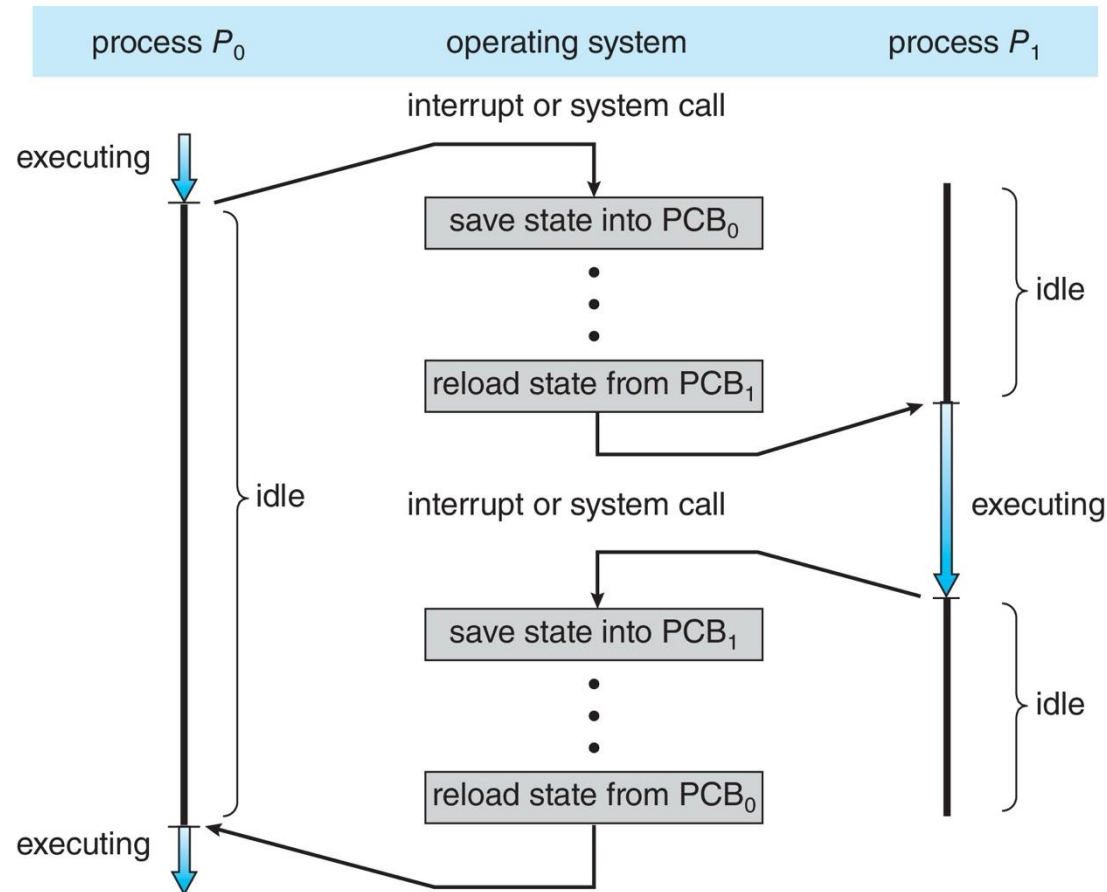


CPU Switch From Process to Process

A **context switch** occurs when the CPU switches from one process to another.

Scheduling decides *who*,
context switching decides *how*.

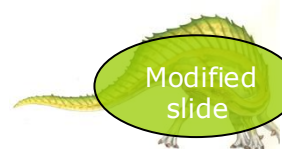
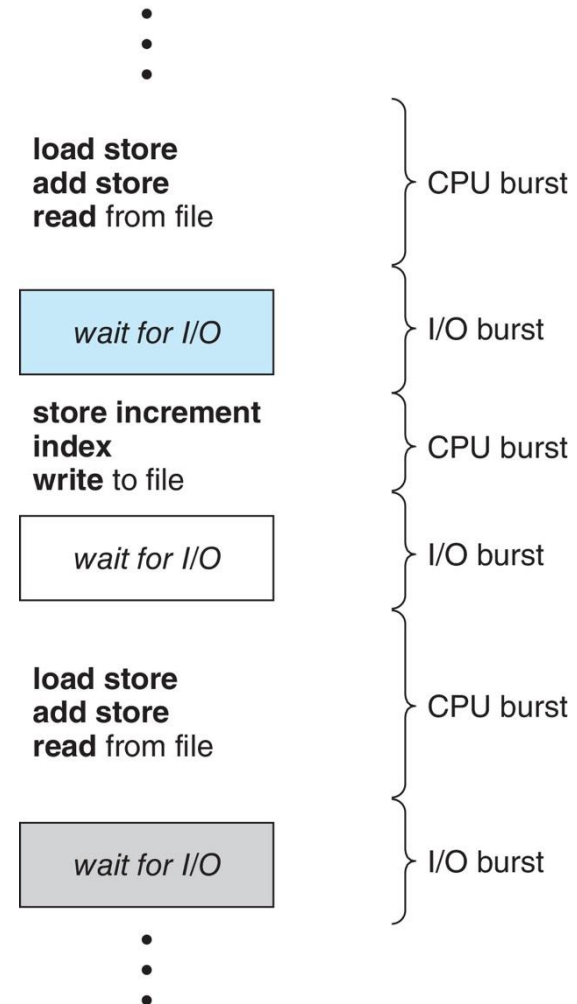
process state
process number
program counter
registers
memory limits
list of open files
...





Basic Concepts

- Obtain **Maximum** CPU utilization with multiprogramming
- CPU-I/O Burst Cycle – Process execution consists of a **cycle** of CPU execution and I/O wait
- **CPU burst** followed by **I/O burst**
- Observed CPU usage over time is of main concern

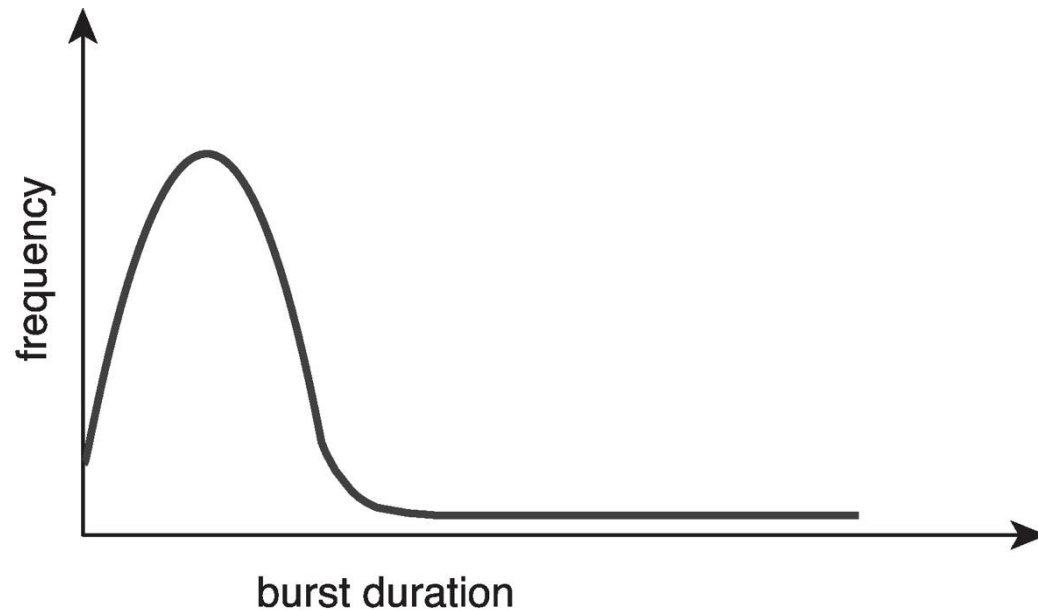




Histogram of CPU-burst Times

Large number of short bursts

Small number of longer bursts





CPU Scheduler

- The **CPU scheduler** selects one process from the ready set and allocates a CPU core to it. Queue may be ordered in various ways
- CPU scheduling **decisions** are involved in the following state transitions:
 1. Switches from running to waiting state
 2. A running process is preempted (running -> ready)
 3. Switches from ready to running state
 4. Terminates
- For situations 1 and 4, the running process must leave the CPU, so another ready process (if any) must be selected.
- For situations 2 and 3, however, there is a choice.

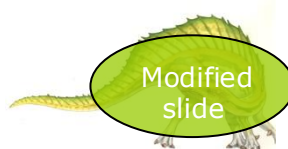
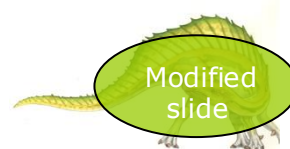
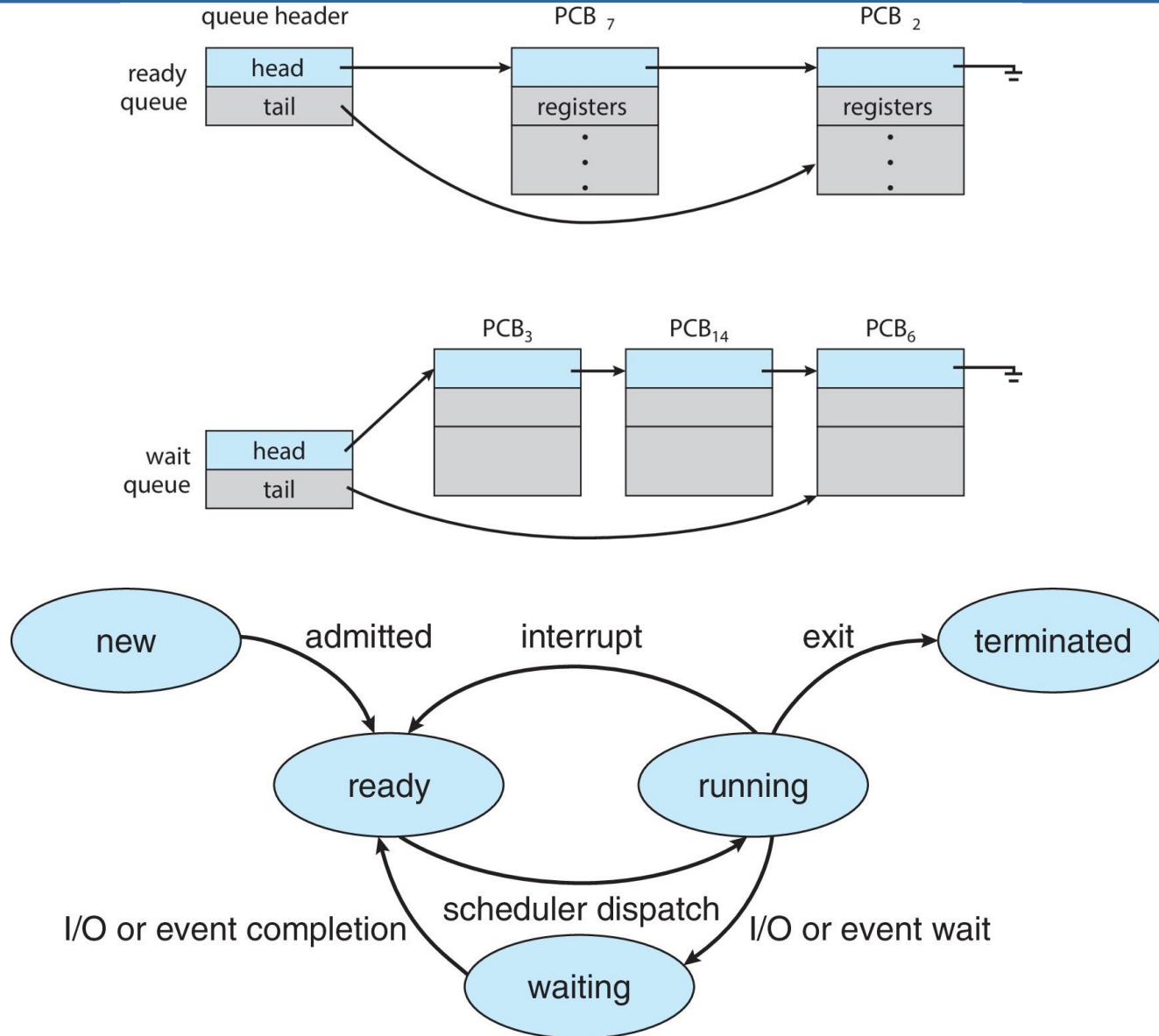




Diagram of Process State

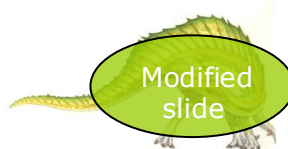


Modified
slide



Preemptive and Nonpreemptive Scheduling

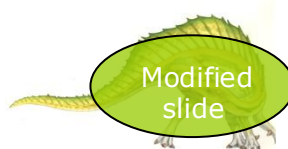
- Under **Nonpreemptive** scheduling, once the CPU has been allocated to a process, the process keeps the CPU until it releases it either by **terminating** or by switching to the **waiting** state.
- Otherwise, it is **preemptive**.
- Virtually all modern general-purpose operating systems including Windows, MacOS, Linux, use preemptive scheduling for normal processes.





Preemptive Scheduling and Race Conditions

- Preemptive scheduling increases the likelihood of **race conditions** when data are shared among concurrent processes.
- Consider the case of two processes that share data. While one process is updating the data, it is preempted so that the second process can run. The second process then tries to read the data, which is in an inconsistent state.
- This issue will be explored more in detail later...





Race Condition

- **counter++** could be implemented as

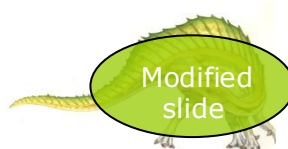
```
register1 = counter
register1 = register1 + 1
counter = register1
```

- **counter--** could be implemented as

```
register2 = counter
register2 = register2 - 1
counter = register2
```

- Consider this execution interleaving with “count = 5” initially:

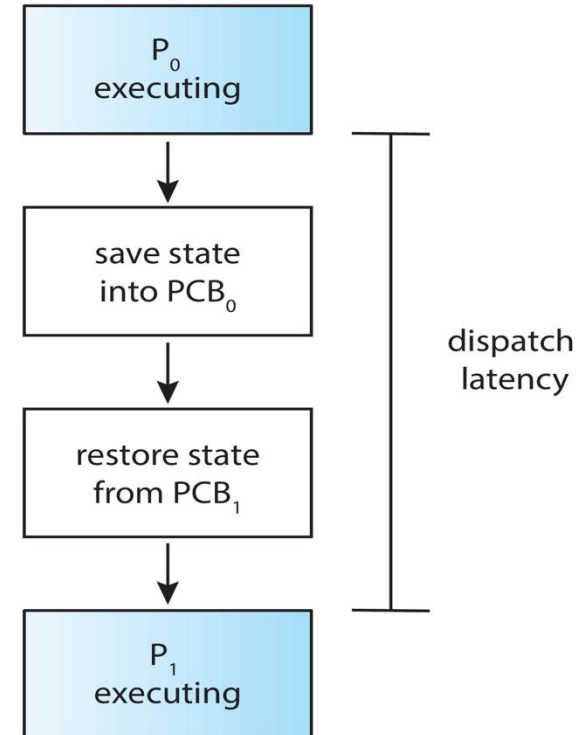
S0: producer execute	register1 = counter	{register1 = 5}
S1: producer execute	register1 = register1 + 1	{register1 = 6}
S2: consumer execute	register2 = counter	{register2 = 5}
S3: consumer execute	register2 = register2 - 1	{register2 = 4}
S4: producer execute	counter = register1	{counter = 6}
S5: consumer execute	counter = register2	{counter = 4}





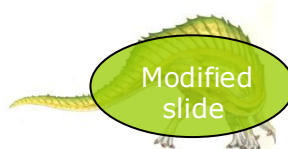
Dispatcher

- **Dispatch latency** – the time it takes for the system to stop one process and start executing the process selected by the scheduler.
- **Dispatch involves** giving control of the CPU to the process selected by the CPU scheduler; this involves:
 - Switching context
 - Switching to user mode
 - Jumping to the proper location in the user program to restart that program





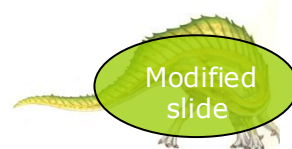
Let's look at basic schedulers ... and how to measure their performance





Scheduling Criteria

- **CPU utilization** – keep the CPU as busy as possible
 - **Throughput** – number of processes that complete their execution per time unit
 - **Turnaround time** – total time from process submission to process completion (includes waiting, execution, and I/O)
 - **Waiting time** – amount of time a process spends in the ready queue
 - **Response time** – amount of time it takes from when a request was submitted until the first response is produced.
-
- Max CPU utilization
 - Max throughput
 - Min turnaround time
 - Min waiting time
 - Min response time
- In practice, improving one metric often worsens another...





First- Come, First-Served (FCFS) Scheduling

<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

- Suppose that the processes arrive in the order: P_1 , P_2 , P_3
The Gantt Chart for the schedule is:



- **Waiting time** for $P_1 = 0$; $P_2 = 24$; $P_3 = 27$
- Average waiting time: $(0 + 24 + 27)/3 = 17$





FCFS Scheduling (Cont.)

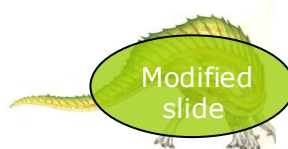
Suppose that the processes arrive in the order:

$$P_2, P_3, P_1$$

- The Gantt chart for the schedule is:



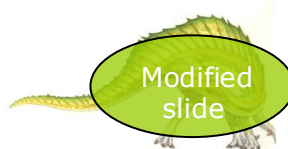
- Waiting time for $P_1 = 6$; $P_2 = 0$; $P_3 = 3$
- Average waiting time: $(6 + 0 + 3)/3 = 3$
- **Better** than previous case i.e. lower average waiting time
- **Convoy effect** - short processes delayed by a long CPU-bound process.





Shortest-Job-First (SJF) Scheduling

- Associate with each process the length (estimate) of its next CPU burst
 - schedule the process with the shortest estimated CPU burst
- SJF is optimal – (in the sense that) it gives minimum average waiting time for a given set of processes with known burst times.
- Preemptive version a.k.a. **shortest-remaining-time-first** (discussed in a moment)
- How do we determine the length of the next CPU burst?
 - ~~• Could ask the user~~
 - Estimate

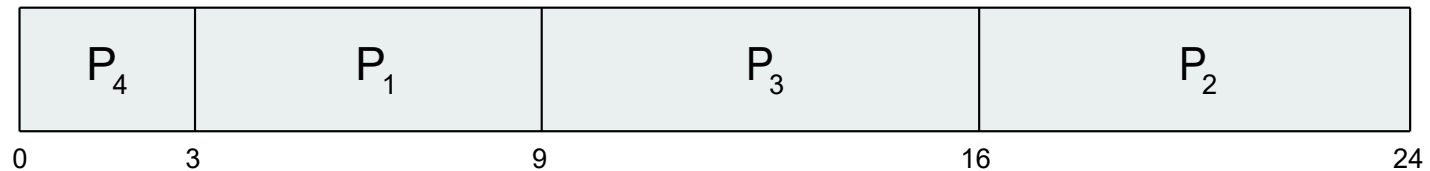




Example of SJF

<u>Process</u>	<u>Burst Time</u>
P_1	6
P_2	8
P_3	7
P_4	3

- SJF scheduling chart



- Average **waiting time** = $(3 + 16 + 9 + 0) / 4 = 7$
- Waiting times: $P_1 = 3$, $P_2 = 16$, $P_3 = 9$, $P_4 = 0$





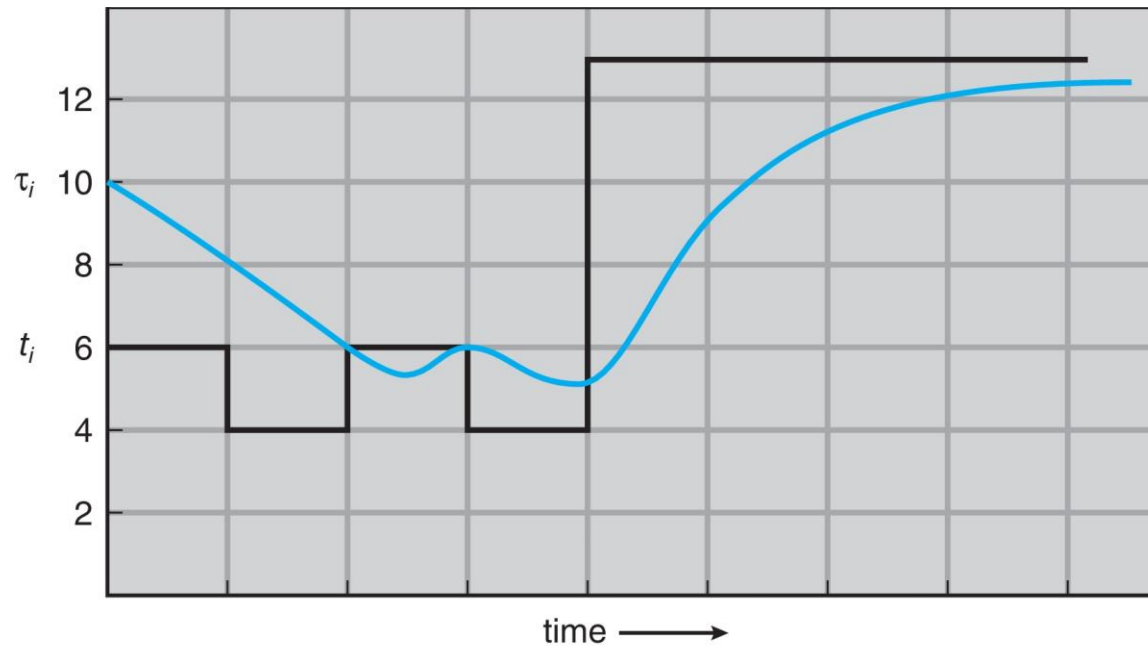
Determining Length of Next CPU Burst

- Can only estimate the length – should be similar to the previous one
 - Then pick process with shortest predicted next CPU burst
- Can be done by using the length of previous CPU bursts, using averaging
 1. t_n = actual length of n^{th} CPU burst
 2. τ_{n+1} = predicted value for the next CPU burst
 3. $\alpha, 0 \leq \alpha \leq 1$
 4. Define :
- Commonly, α set $\tau_{n+1} = \alpha t_n + (1 - \alpha)\tau_n$.





Prediction of the Length of the Next CPU Burst



CPU burst (t_i)	6	4	6	4	13	13	13	...	
"guess" (τ_i)	10	8	6	6	5	9	11	12	...





Examples of Exponential Averaging

- $\alpha = 0$
 - $\tau_{n+1} = \tau_n$
 - Recent history does not count
- $\alpha = 1$
 - $\tau_{n+1} = \alpha t_n$
 - Only the actual last CPU burst counts

- If we expand the formula, we get:

$$\begin{aligned}\tau_{n+1} = & \alpha t_n + (1 - \alpha)\alpha t_{n-1} + \dots \\ & + (1 - \alpha)^j \alpha t_{n-j} + \dots \\ & + (1 - \alpha)^{n+1} \tau_0\end{aligned}$$

- Since both α and $(1 - \alpha)$ are less than or equal to 1, each successive term has less weight than its predecessor



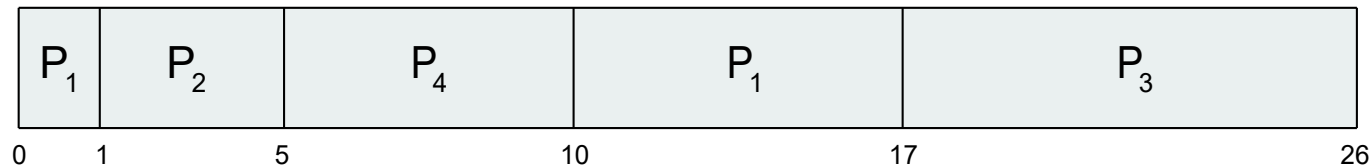


Example of Shortest-remaining-time-first

- **SRTF**: Shortest-remaining-time-first algorithm: the preemptive version of shortest job first
- Now we add the concepts of **varying arrival times** and preemption to the analysis

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0	8
P_2	1	4
P_3	2	9
P_4	3	5

- *Preemptive* SJF Gantt Chart

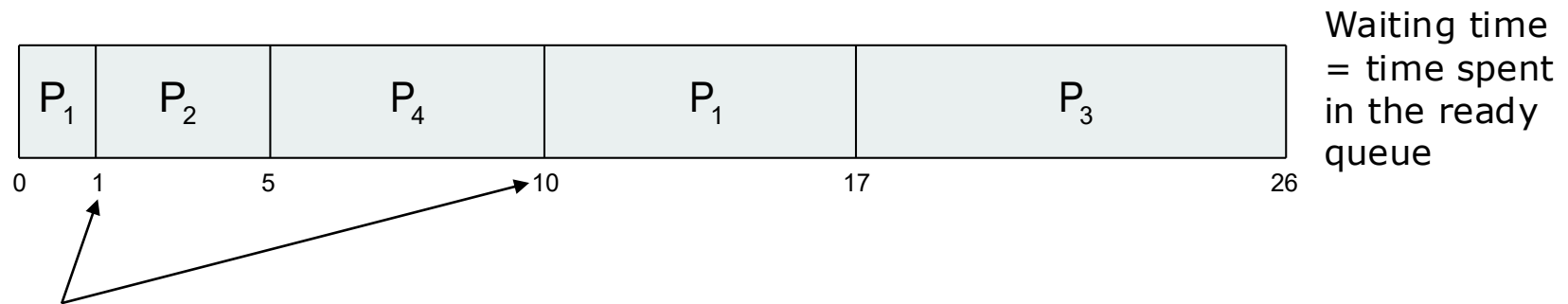


- Average waiting time = $[(10-1)+(1-1)+(17-2)+(5-3)]/4 = 26/4 = 6.5$

Modified
slide



Example of Shortest-remaining-time-first



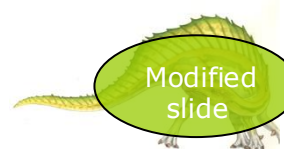
10-1 (P1)
arrived at 0, executed for 1, and then spent 9 on waiting on ready queue

1-1 (P2)
arrived at 1, executed for 4, and then spent 0 on waiting on ready queue

5-3 (P4)
arrived at 3, had to wait 2 for P2 to finish (as it had shorter burst time), and then it ran at 5

17-2 (P3)
arrived at 2, other job (P1) had a shorter burst time, so it has to wait 15 to execute

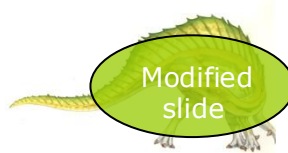
$$\text{Average waiting time} = [(10-1)+(1-1)+(17-2)+(5-3)]/4 = 26/4 = 6.5$$





What about?

- **Turnaround time** – amount of time to execute a particular process
- **Waiting time** – amount of time a process has been waiting in the ready queue
- **Response time** – amount of time it takes from when a request was submitted until the first response is produced.



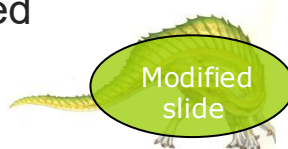


Round Robin (RR)

- Each process gets a small unit of CPU time (**time quantum** q), usually 10-100 milliseconds. After this time has elapsed, the process is preempted and added to the end of the ready queue.
- If there are n processes in the ready queue and the time quantum is q , then each process gets $1/n$ of the CPU time in chunks of at most q time units at once. No process waits more than $(n-1) * q$ time units.
- **Timer** interrupts every quantum to schedule next process
- Performance
 - q large $\Rightarrow$ FIFO
 - q small $\Rightarrow q$ must be large with respect to context switch, otherwise overhead is too high

Linux uses something a little more complex than this for scheduling, but the concept of interruption, time quantum and RR applies

In linux there's a **scheduler_tick()** that gets called periodically

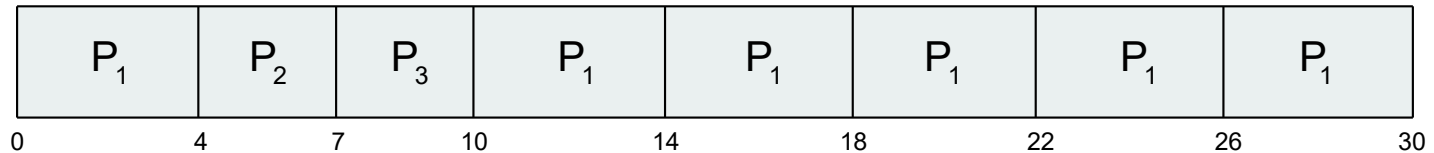




Example of RR with Time Quantum = 4

<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

- The Gantt chart is:

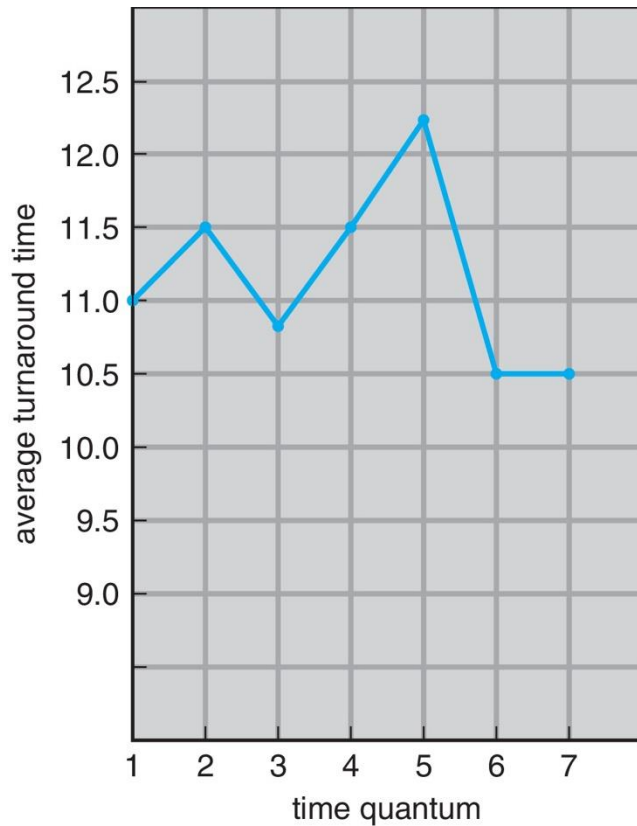


- Typically, higher average turnaround than SJF, but better **response**
- q should be large compared to context switch time
 - q usually 10 milliseconds to 100 milliseconds,
 - Context switch < 10 microseconds

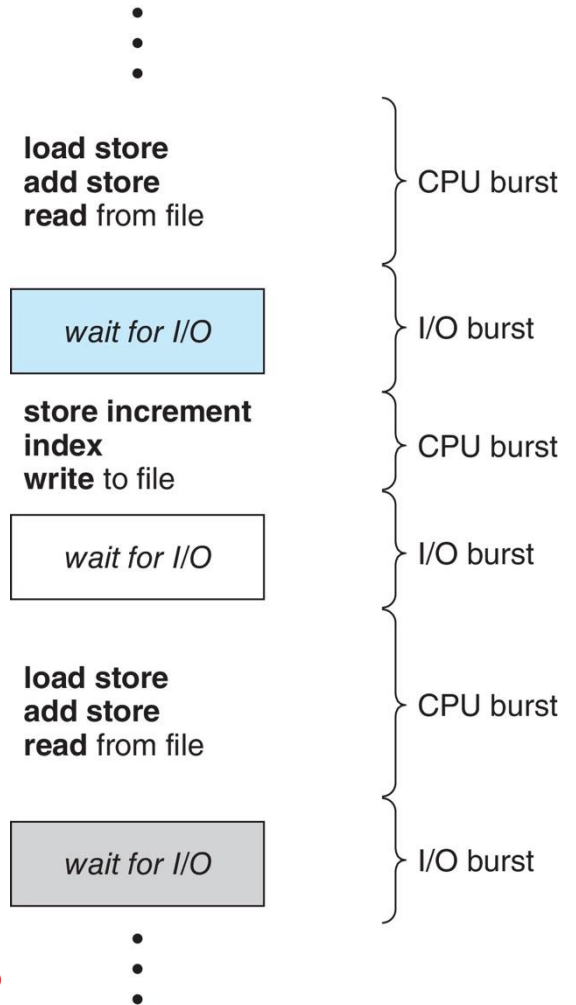




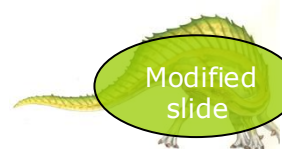
Turnaround Time Varies With The Time Quantum



process	time
P_1	6
P_2	3
P_3	1
P_4	7



Rule of thumb: 80%
CPU bursts should be
shorter than q

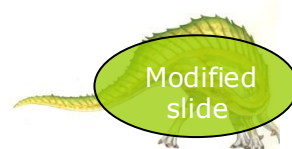




Priority Scheduling

Checkout...
nice and renice
commands in Linux
man(nice)

- A priority number (integer) is associated with each process
- The CPU is allocated to the process with the highest priority
 - Preemptive
 - Nonpreemptive
- SJF is *akin* priority scheduling where priority is the inverse of predicted next CPU burst time
- Problem $\equiv$ **Starvation** – low priority processes may never execute
- One Solution is ... $\equiv$ **Aging** – as time progresses increase the priority of the process

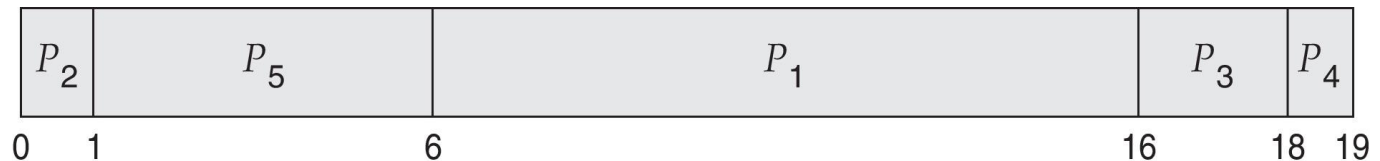




Example of Priority Scheduling

<u>Process</u>	<u>Burst Time</u>	<u>Priority</u>
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

- Priority scheduling Gantt Chart



- Average waiting time = 8.2

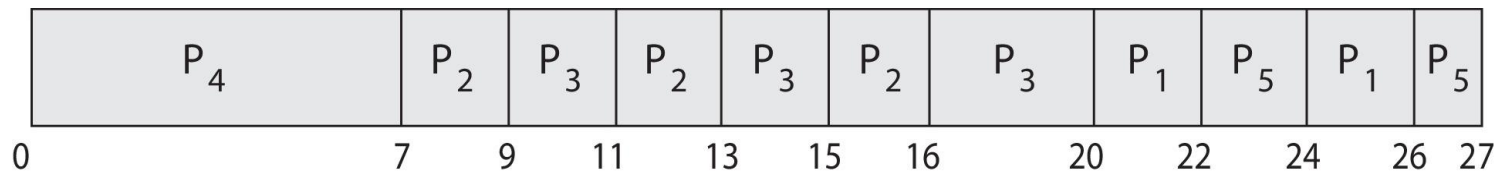




Priority Scheduling w/ Round-Robin

<u>Process</u>	<u>Burst Time</u>	<u>Priority</u>
P_1	4	3
P_2	5	2
P_3	8	2
P_4	7	1
P_5	3	3

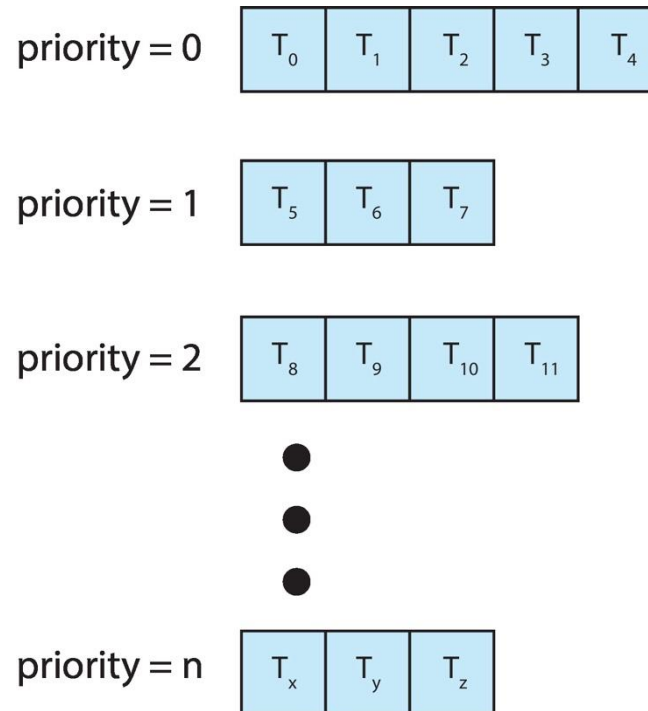
- Run the process with the highest priority. Processes with the same priority run round-robin
- Gantt Chart with time quantum = 2





Multilevel Queue

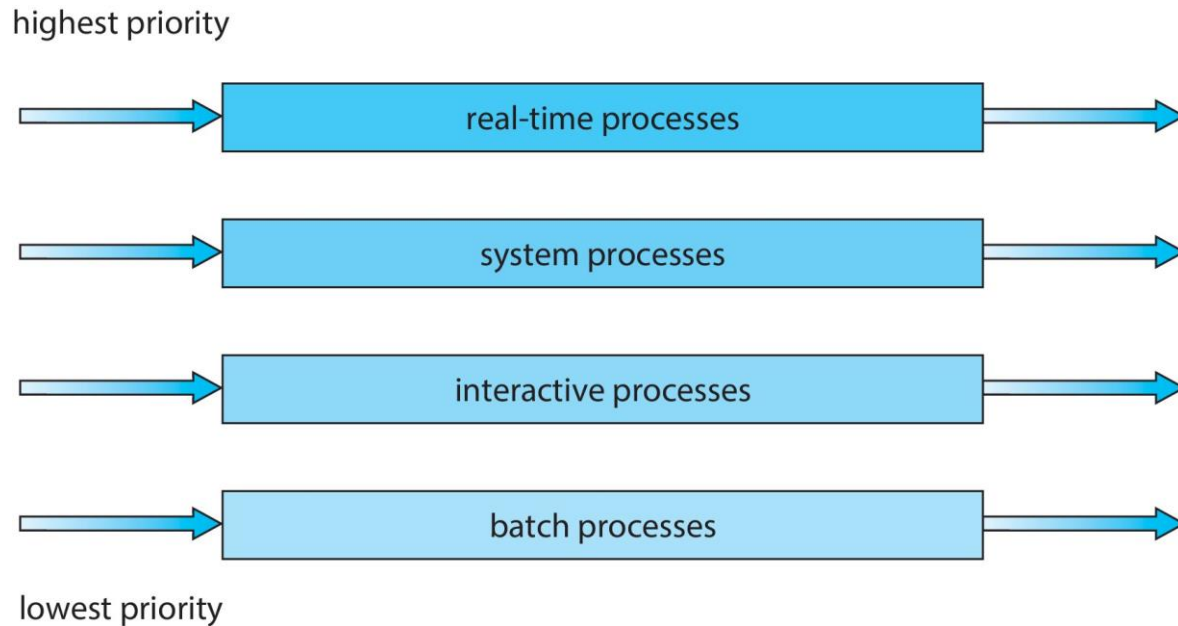
- With priority scheduling, have separate queues for each priority.
- Schedule the process in the highest-priority queue!





Multilevel Queue

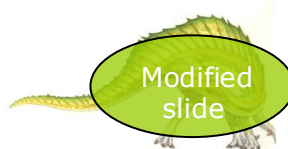
- Prioritization based upon process type





Multilevel Feedback Queue

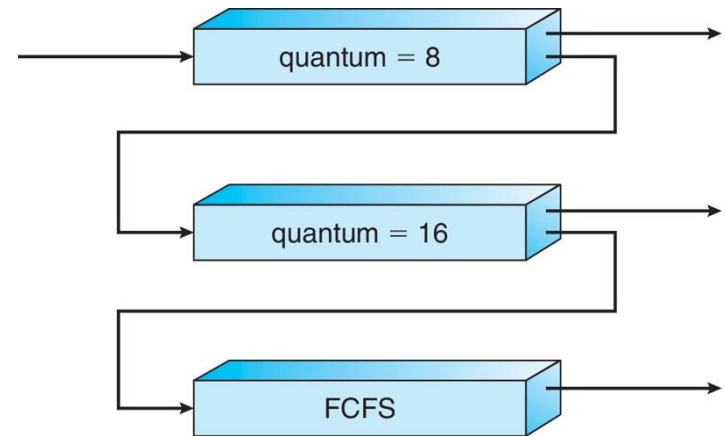
- A process can **move between the various queues**.
- **Multilevel-feedback-queue scheduler** defined by the following parameters/features:
 - Number of queues
 - **Different** scheduling algorithms for each queue
 - Method used to determine when to **upgrade** a process
 - Method used to determine when to **demote** a process
 - Method used to determine which queue a process will enter when that process needs service
- Aging can be implemented using multilevel feedback queue



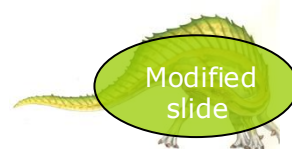


Example of Multilevel Feedback Queue

- Three queues:
 - Q_0 – RR with time quantum 8 milliseconds
 - Q_1 – RR time quantum 16 milliseconds
 - Q_2 – FCFS, etc
- Scheduling
 - A new process enters queue Q_0 which is served in RR
 - ▶ When it gains CPU, the process receives 8 milliseconds
 - ▶ If it does not finish in 8 milliseconds, the process is moved to queue Q_1
 - At Q_1 job is again served in RR and receives 16 additional milliseconds
 - ▶ If it still does not complete, it is preempted and moved to queue Q_2



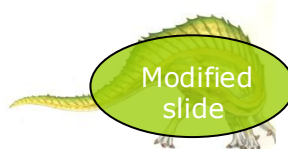
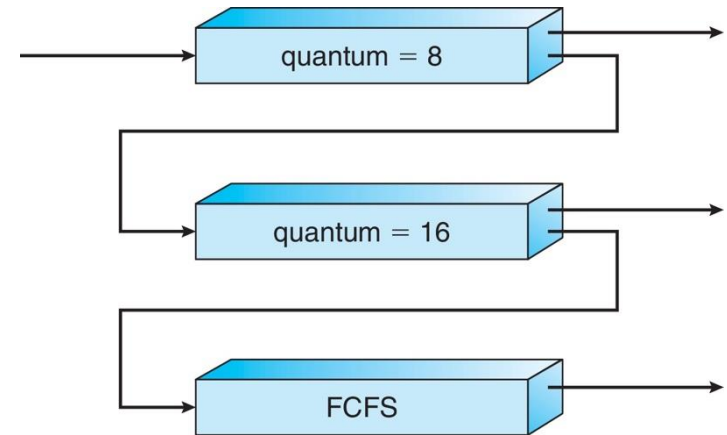
Problem... Processes in the low priority queues can starve, solution may be to boost priority after X time has passed





Example of Multilevel Feedback Queue

- Strategies like this:
 - Reduce feedback time
 - More flexible than plain MQs (starvation)
- Downside is the complexity





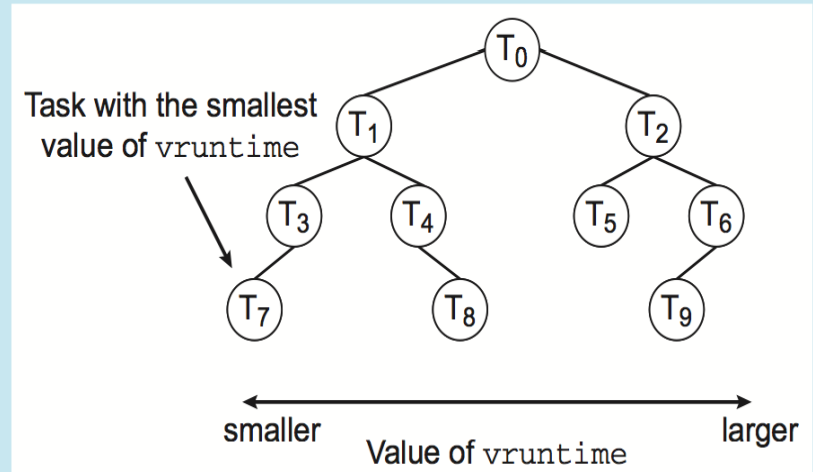
In Linux

- Scheduler_tick() in

<https://github.com/torvalds/linux/blob/v5.5/kernel/sched/core.c#L3586-L3614>

- `/proc/sys/kernel/sched_rr_timeslice_ms`

The Linux CFS scheduler provides an efficient algorithm for selecting which task to run next. Each runnable task is placed in a red-black tree—a balanced binary search tree whose key is based on the value of `vruntime`. This tree is shown below:

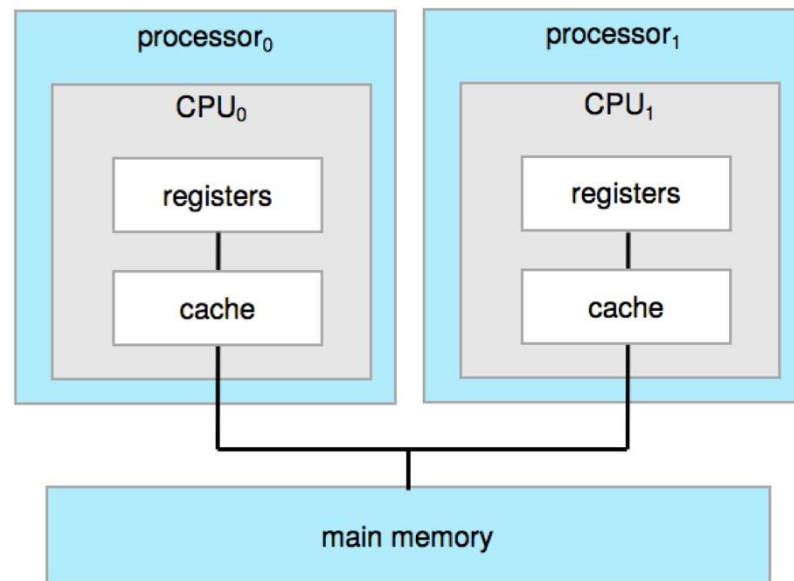
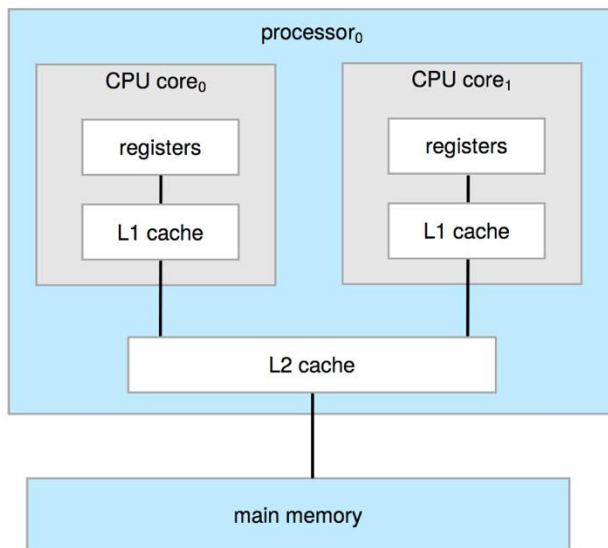


When a task becomes runnable, it is added to the tree. If a task on the tree is not runnable (for example, if it is blocked while waiting for I/O), it is removed. Generally speaking, tasks that have been given less processing time (smaller values of `vruntime`) are toward the left side of the tree, and tasks that have been given more processing time are on the right side. According to the properties of a binary search tree, the leftmost node has the smallest key value, which for the sake of the CFS scheduler means that it is the task with the highest priority. Because the red-black tree is balanced, navigating it to discover the leftmost node will require $O(\lg N)$ operations (where N is the number of nodes in the tree). However, for efficiency reasons, the Linux scheduler caches this value in the variable `rb_leftmost`, and thus determining which task to run next requires only retrieving the cached value.



Multiple-Processor Scheduling

- CPU scheduling more complex when multiple CPUs are available
- Multiprocess may be any one of the following architectures:
 - Multicore CPUs
 - Multithreaded cores
 - NUMA systems
 - Heterogeneous multiprocessing





Multiple-Processor Scheduling – Processor Affinity

- When a thread/process has been running on one processor, the cache contents of that processor stores the memory accesses by that thread.
- We refer to this as a thread having **affinity** for a processor (i.e., “processor affinity”)
- Load balancing may affect processor affinity as a thread may be moved from one processor to another to balance loads, yet that thread loses the contents of what it had in the cache of the processor it was moved off of.
- **Soft affinity** – the operating system attempts to keep a thread running on the same processor, but no guarantees.
- **Hard affinity** – allows a process to specify a set of processors it may run on.

